# TCP Congestion Control (Part III)

## CEN 223 – Internet Communication

Assoc. Prof. Dr. Fatih ABUT
Department of Computer Engineering
Çukurova University

# Contents

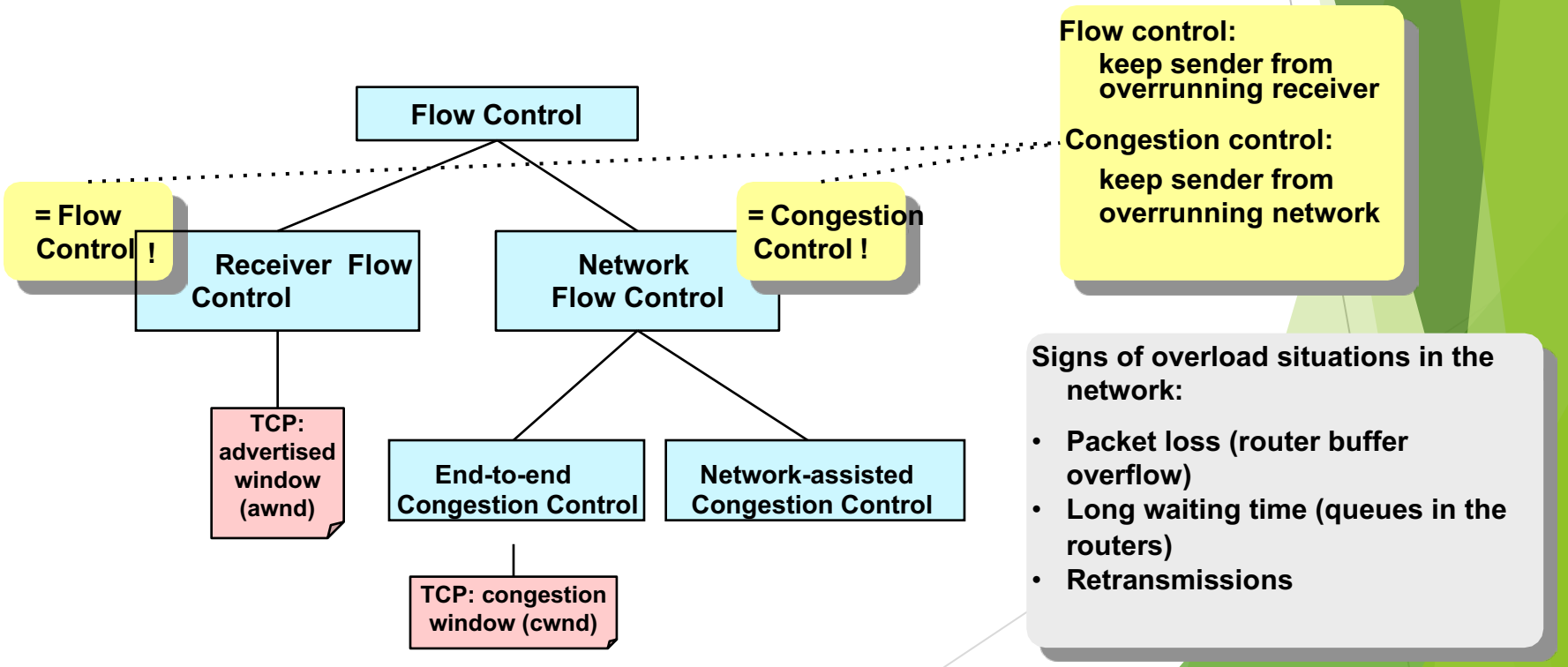
- Introduction
- Slow-start
- Congestion avoidance
  - Additive Increase
  - Multiplicative Decrease
- Fast retransmit
- Fast recovery
- TCP Fairness

# Definition of terms: congestion and flow control

**Definition (according to Steven Low):**

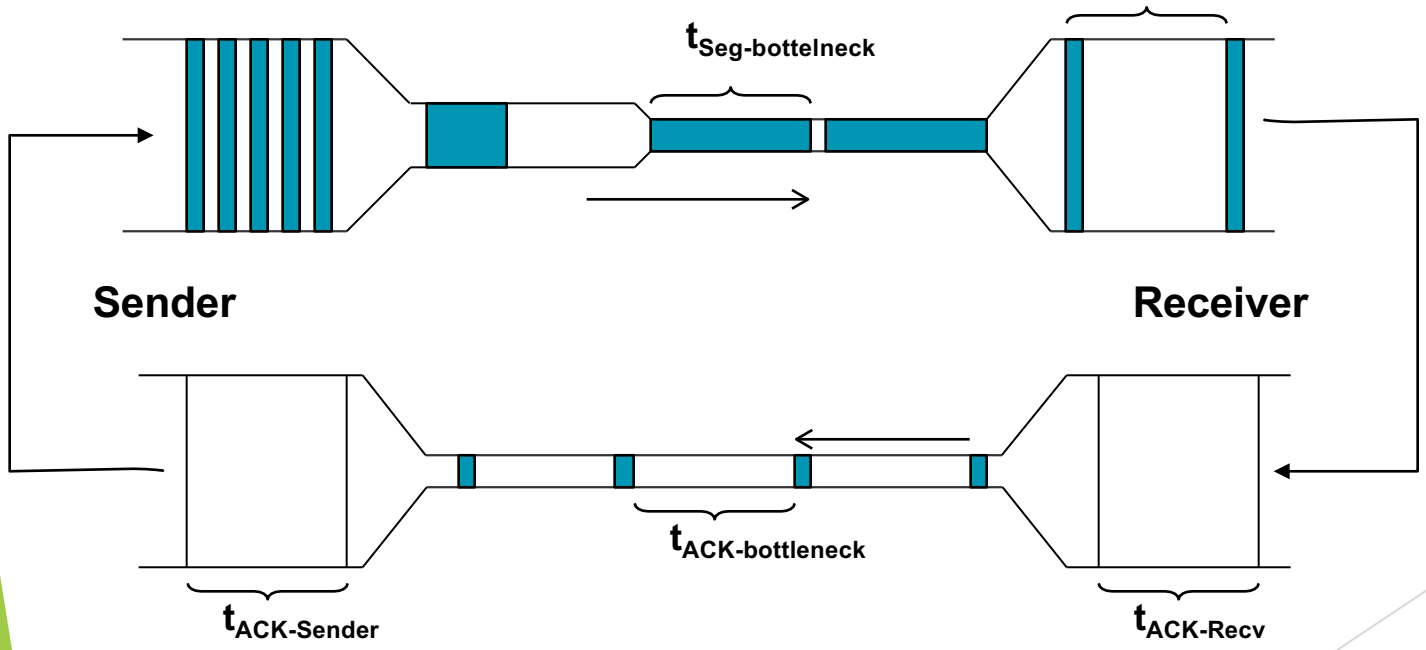
With flow control we refer to all methods for the efficient use of bandwidth (of a connectionless network)

With window flow control we refer to flow control mechanisms that are based on window sizes (= buffer sizes).



# Principle: Self-Clocking

**Equilibrium through "Conservation of Packets"**




**TCP Selfclocking -> Adaptation of the transmission rate to the slowest connection part**

**Key question: Congestion Avoidance:**

**How is Equilibrium ensured?**

# TCP Congestion Control

**TCP tries to achieve the following goals through congestion control:**

**(1) high usage of the network (-> high equilibrium)**
**(2) as no overload as possible (-> stable equilibrium)**
**(3) fair distribution of the available bandwidth (-> fair equilibrium)**

**TCP uses two flow control mechanisms for this purpose:**

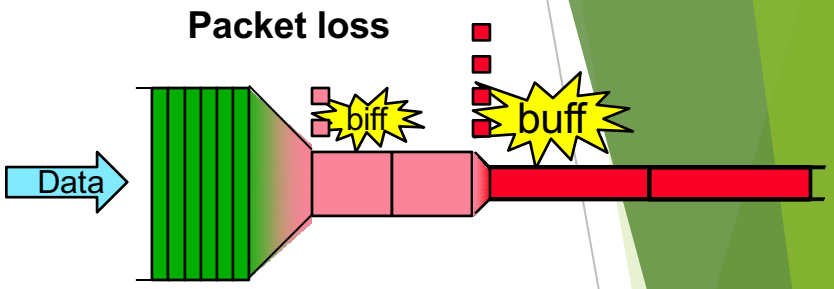
- **Receiver Flow Control**
  - **Avoidance of overload at the receiver**
  - **Controlled by the receiver**
  - **Window size used: AdvertisedWindow (awnd)**
- **Network Flow Control**
  - **Collects information about the resilience of the network (loss, delay, hints) -> (1)**
  - **Avoid overloading the network -> (2)**
  - **Controlled by the transmitter**
  - **Window size used: CongestionWindow (cwnd)**

**Window effectively used: wnd = min(awnd, cwnd)**

# Critical phases, equilibrium and algorithms

**Start of the TCP transmission**

- **Slow Start (Build an equilibrium)**




**Maintaining Equilibrium**

- **Setting the RTO (Jacobson/Karel Algorithm)**
- **Maximize utilization (–> Additive Increase)**

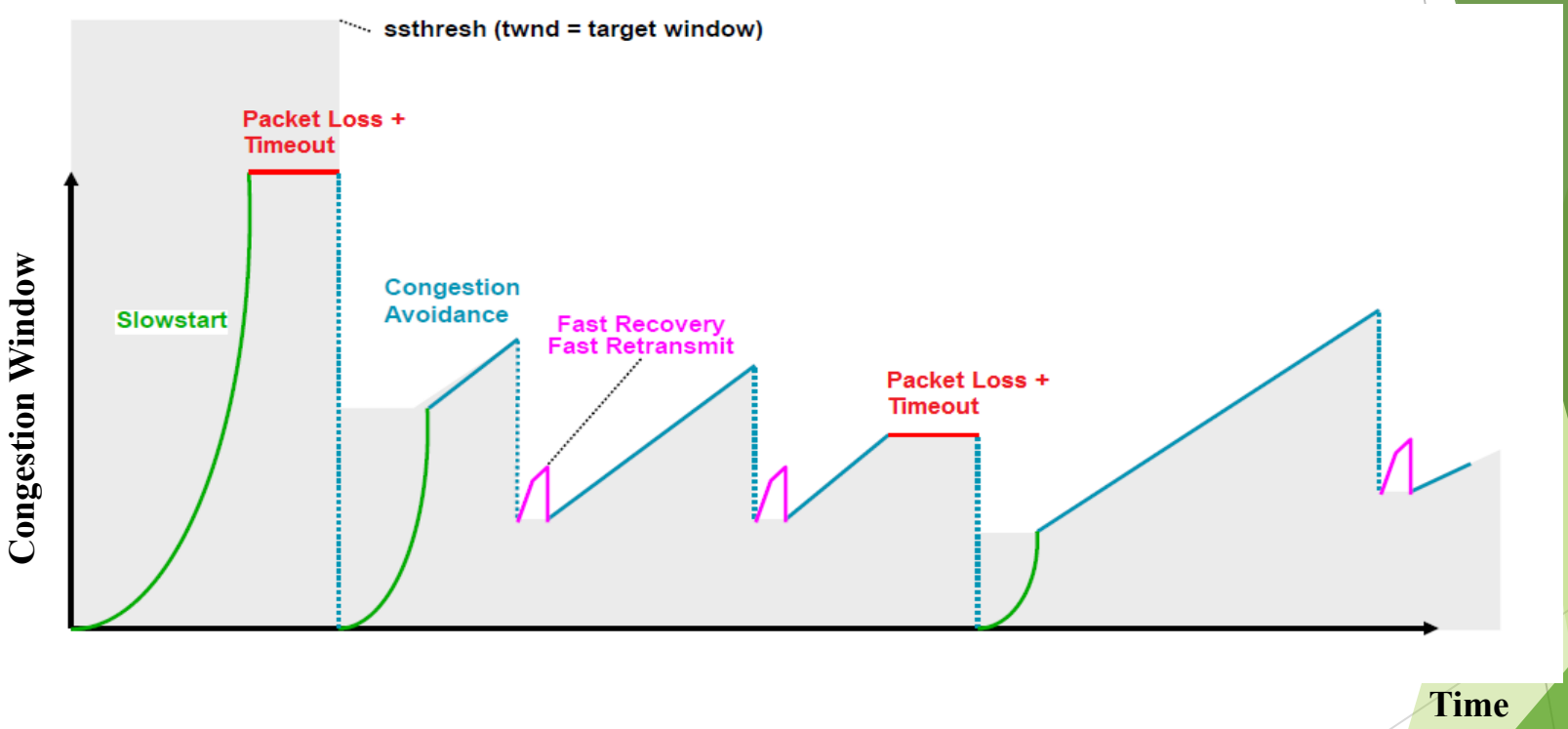
**Response to signs of overload**

- **Stabilization of the situation (–> Multiplicative Decrease, Karn-Algo.)**
- **Ensuring a continuous, uninterrupted flow of data (–> Fast Retransmit, –>Fast Recovery, ....)**

**Fairness of resource allocation**

- **Automatically (?) through AIMD (= Additive Increase / Multiplicative Decrease)**

# Typical “Sawtooth" Pattern



# Slow Start (1)

The aim of slow start :

- Rapid introduction of the TCP connection to the Equilibrium strategy:
- Progressive testing of the load capacity of the route between the sender and receiver (up to the overload)
- Packet losses (-> retransmission time out) are evaluated as an indicator of an overload situation. (This is critical for mobile TCP -> low transmission rate)
- Calming of the network after the occurrence of an overload situation (emptying the router queues at the bottleneck)

Application:

- When starting a connection
- After the retransmission timer has expired
- After a long passive TCP phase

# Slow Start (2)

**Slow Start**

**cwnd = SMSS**

**FOR EACH received(ACK)**

**cwnd = cwnd + SMSS**

**UNTIL cwnd >= twnd OR RTO**

- SMSS Sender Maximum Segment Size

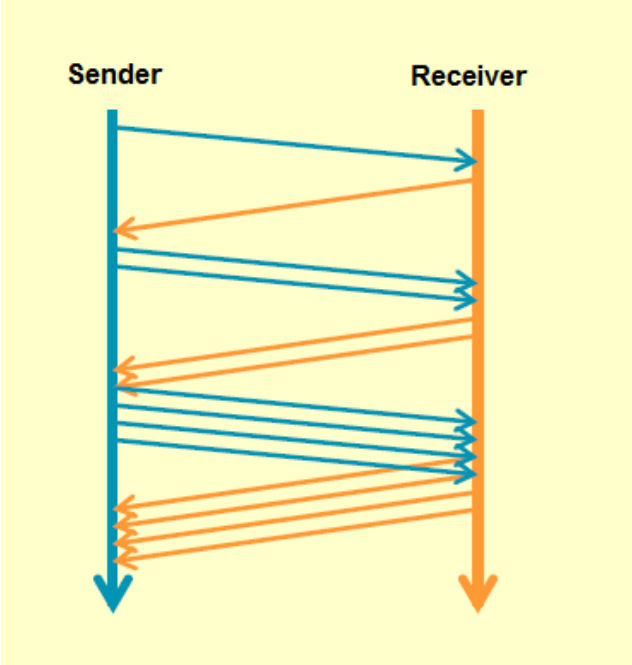
largest segment that a transmitter can send

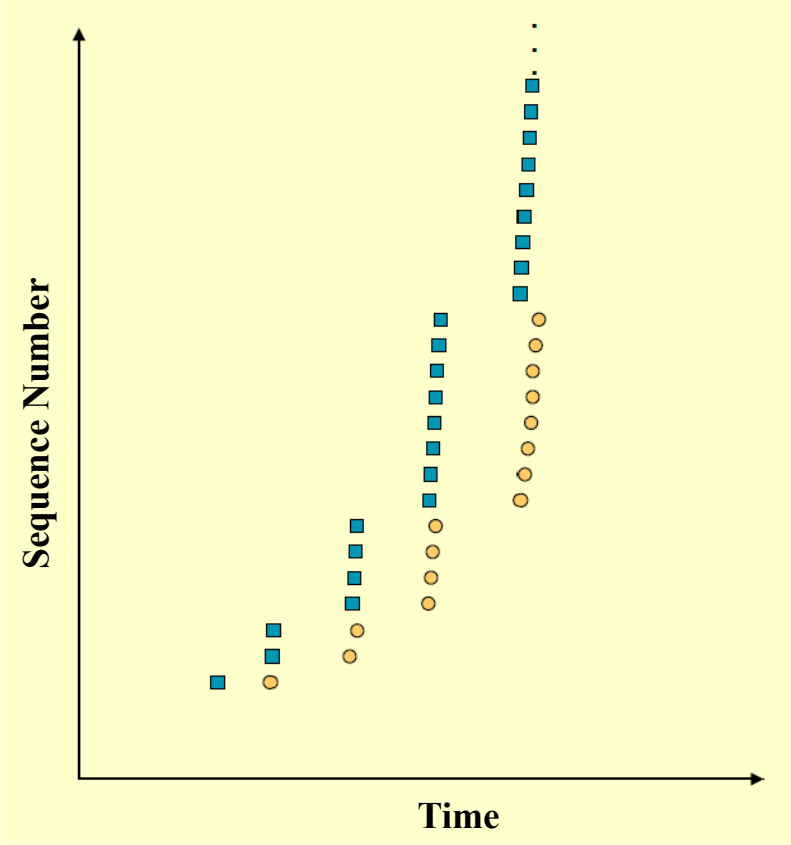
(536 bytes according to RFC 1122, 1024 bytes for most implementations).

- The start value for cwnd can be set to up to 2 SMSS (according to RFC 2581)
- For the first slow start, the target window (twnd = ssthresh = slow start threshhold) can be set as high as you want (e.g. to awnd)
- (approximate) growth of cwnd as a function of time measured in RTT:

$$cwnd\ (t) = \sum_{i=0}^{t} 2^i = 2^{t+1} - 1$$

# Slow Start (3)



# Congestion Avoidance

Aim:

- Best possible utilization of the available bandwidth (greedy), but avoid overload if possible.

Strategy:

- Continuous linear increase in the cwnd as long as no overload situation occurs (adding 1 SMSS per RTT)
  –> Additive Increase

- Halving the cwnd in the event of an overload situation. An overload situation is assumed, e.g. after the retransmission timer has expired.
  –> Multiplicative Decrease

- AIMD = Additive Increase / Multiplicative Decrease

# Congestion Avoidance Algorithm

**Congestion Avoidance**

**UNTIL** LossEvent

**FOR EACH** received(ACK)

cwnd = cwnd + SMSS * (SMSS/cwnd)

**ENDFOR**

**ENDUNTIL**

twnd := cwnd/2

**IF** intelligent

Perform Fast Retransmit or Fast Recovery

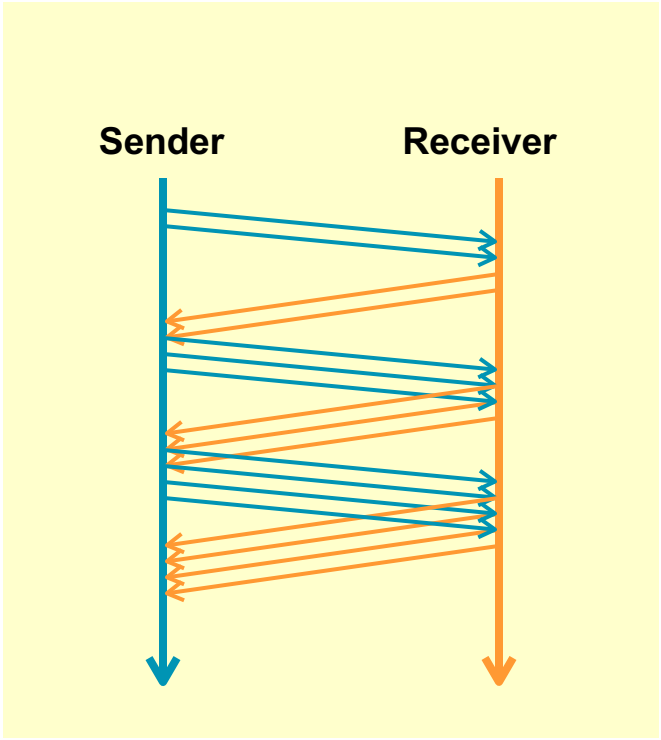
**ELSE**

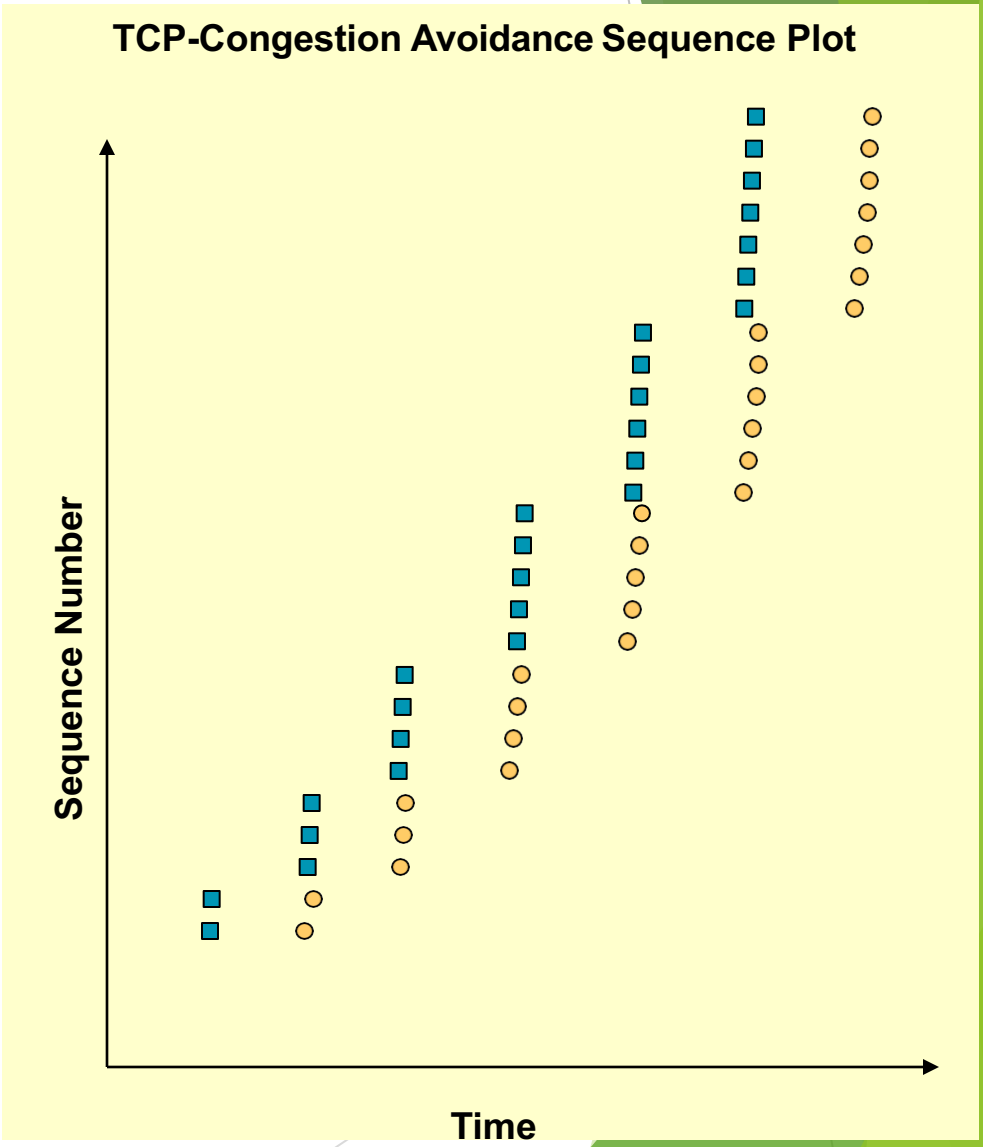
cwnd :=1
Perform SlowStart

**ENDIF**

**Loss Event can be triggered by three dupACKs or RTO**

# Illustration of the Congestion Avoidance Algorithm



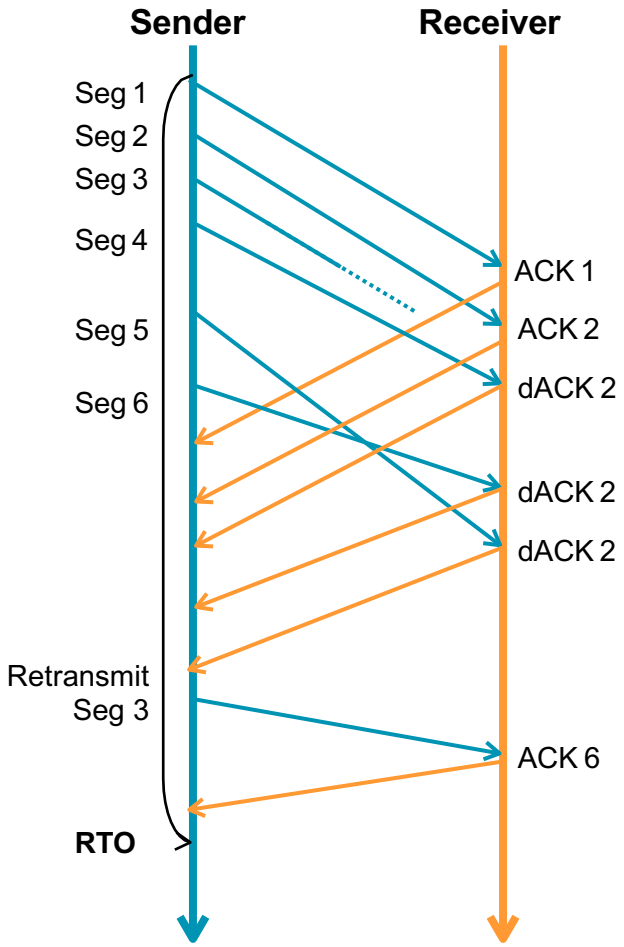
# Fast Retransmit (1)

- Coarse timeouts remained a problem, and Fast retransmit was added with TCP Tahoe.
- Since the receiver responds every time a packet arrives, this implies the sender will see duplicate ACKs.
- Basic Idea: *use **duplicate ACKs** to signal lost packet.*

**Fast Retransmit**

Upon receipt of *three* duplicate ACKs, the TCP Sender retransmits the lost packet.

# Fast Retransmit (2)




**Problem:**

- **Rough setting of the RTO leads to waiting times in the event of packet loss**

**Solution:**

- **In the event of a triple duplicate ACK, retransmission of the missing package without waiting for the retransmission timer to expire**

**New Problem:**

- **A lost packet is a sign of network congestion. Therefore, after Fast Retransmit, a reaction to network overload is necessary.**

**–> Congestion Control**

**Note:**
**Fast Recovery: Algorithm to get data flow after Fast Retransmit**
**SACK (Selective Acknowledgement, RFC2018): Confirmation of the segments actually received**

# Fast Recovery (1)

- Fast recovery was added with TCP Reno.
- Basic idea: When fast retransmit detects three duplicate ACKs, start the recovery process from congestion avoidance region and use ACKs in the pipe to pace the sending of packets.

**Fast Recovery**

**After Fast Retransmit, half cwnd and commence recovery from this point using linear additive increase 'primed' by left over ACKs in pipe.**

# Fast Recovery (according to RFC 2581)

**Fast Recovery**

**AS-SOON-AS #dACK=3**

**twdn := 0.5 * cwdn -- multiplicative decrease**

**cwnd := twnd + 3 * SMSS -- inflating**

**send(LostSegment)**

**END AS-SOON-AS**

**FOR-EACH-ADDITIONAL dACK**

**cwnd := cwnd + SMSS**

**END-FOR-EACH ADDITIONAL**

**AS-SOON-AS regular-ACK**

**cwnd := twnd -- deflating**

**END AS-SOON-AS**

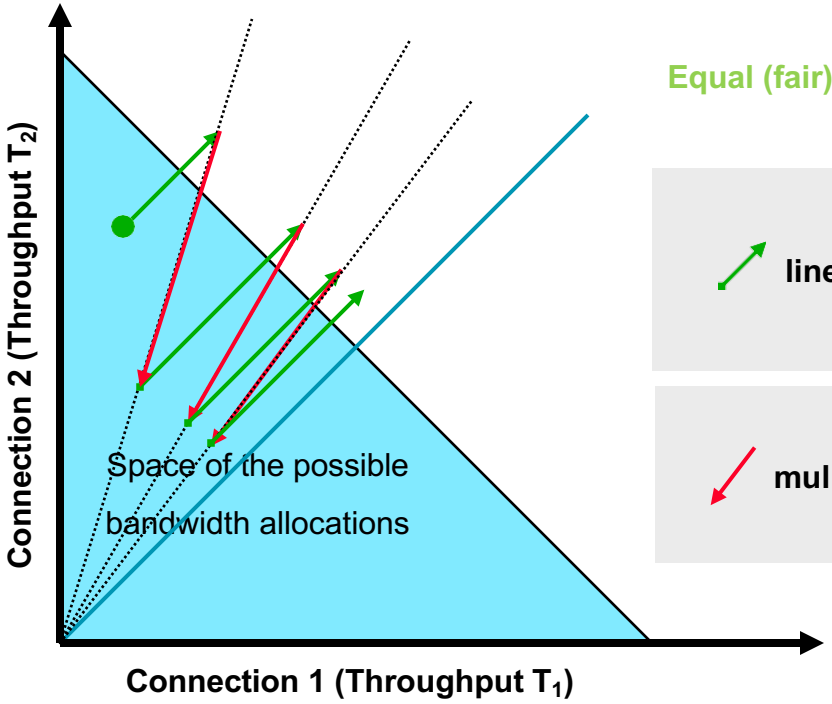
**Strategy:**

Additional opening of the cwnd in accordance with the "Equilibrium concept" is intended to control continuous uninterrupted shutdown of the pipe to the new setpoint value of the twnd.

# TCP Fairness

Two simultaneous TCP sessions

- Additive Increase → Slope 1
- Multiplicative Decrease → Proportional slope




Equal (fair) distribution of bandwidth

linear Increase $\frac{\Delta T_1}{\Delta T_2} = 1$

mulitiplikative decrease $\frac{\Delta T_1}{\Delta T_2} = \frac{T_1}{T_2}$

# UDP: User Datagram Protocol [RFC 768]

- "bare bones", "best effort" transport protocol
- *connectionless:*
  - no handshaking between UDP sender, receiver before packets start being exchanged
  - each UDP segment handled independently of others
- Just provides multiplexing/demultiplexing

**Pros:**

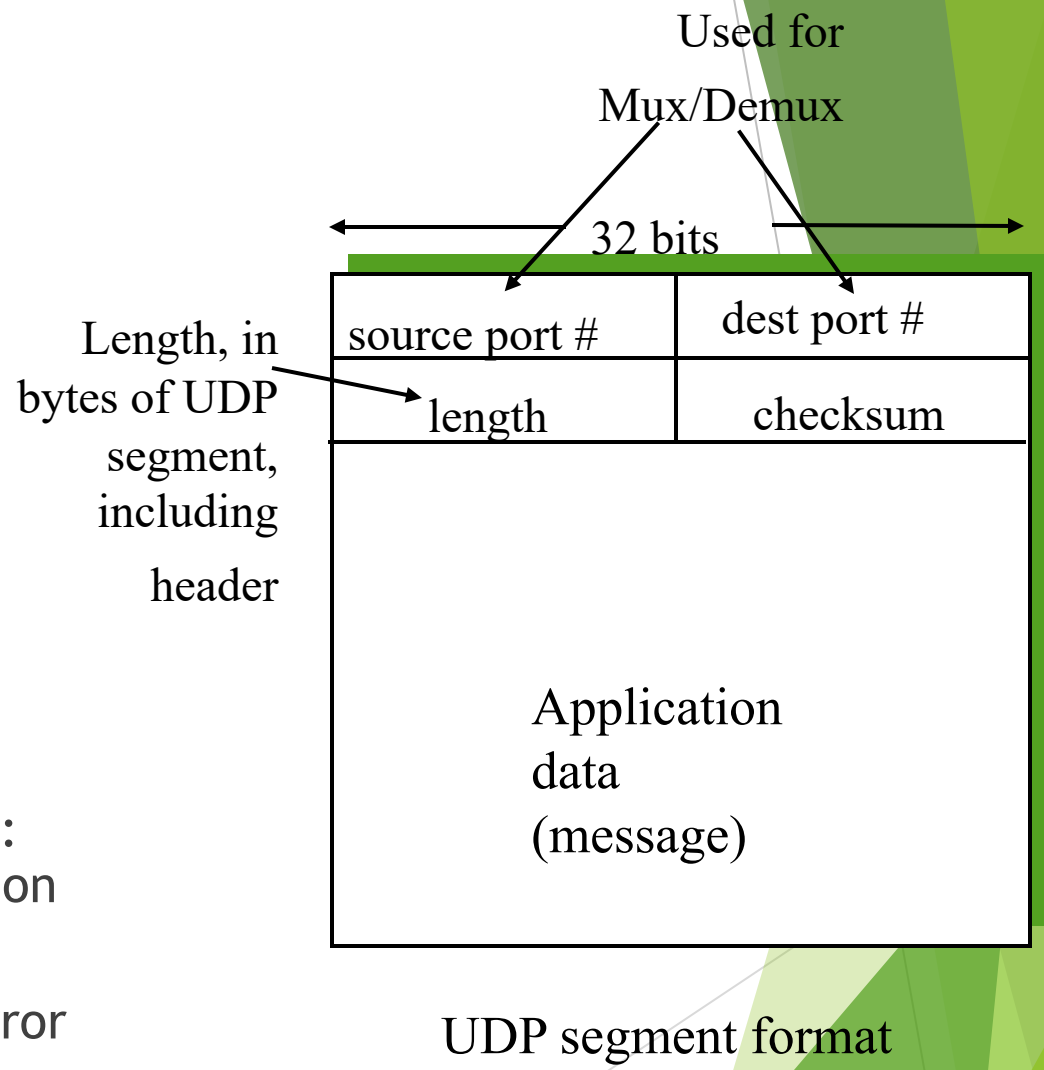
- No connection establishment
  - No delay to start sending/receiving packets
- Simple
  - no connection state at sender, receiver
- Small segment header
  - Just 8 bytes of header

**Cons:**

- "best effort" transport service means, UDP segments may be:
  - lost
  - delivered out of order to app
- no congestion control: UDP can blast away as fast as desired

# UDP more

- often used for streaming multimedia apps
  - loss tolerant
  - rate sensitive
- other UDP uses
  - DNS
  - SNMP
- reliable transfer over UDP: add reliability at application layer
  - application-specific error recovery!




UDP segment format